基于 OpenMP 的 Sobel 图像边缘检测
任务划分、同步机制与并行性能分析

学号：_____ 姓名：_____

（学院、专业及班级：_____）

实验日期：2026 年 10 月 25 日

摘要 针对 Sobel 边缘检测中大量独立的邻域计算，本文从任务分解与共享数据依赖出发，设计串行、静态分块、锁任务池和原子任务池四种实现。各版本采用相同卷积公式及输出规则，将固定的 4096×4096 灰度图划分为 64 个行块，保持每块最多 64 行。静态方案预先分配任务，动态方案分别用锁和原子捕获保护任务编号，像素计算均在临界区外进行。实验固定输入与编译参数，比较 4 线程下的不同设计，以及 1、2、4 线程下的扩展性。每组预热 3 次、正式测量 30 次，并在独立诊断运行中记录任务分配与线程工作时间。串行平均耗时为 57.914 ms，三种 4 线程方案分别为 28.866、19.610 和 21.021 ms。当前云环境下，动态任务分配优于静态分块，但结果不能直接解释为锁操作优于原子操作。本文结合依赖关系、任务所有权与实测诊断讨论并行收益及其适用范围。

关键词 Sobel；OpenMP；任务分解；共享任务池；互斥锁；原子操作；并行效率

1 引言

边缘检测根据局部灰度变化提取轮廓，是图像处理的基础操作。Sobel 算子以两个 3×3 卷积核估计水平和垂直梯度。固定图像中，各输出像素只依赖原图邻域，不依赖其他输出值，因而具有明确的数据并行性。

本作业围绕“针对性地用并行程序解决问题”展开[1]。研究重点不是启动多少线程，而是如何划分工作、安排数据访问、保护确实存在的共享状态，以及如何通过实验判断这些选择是否有效。本文只实现 Sobel，不增加其他图像处理算法；性能实验限定为设计方案对比与线程扩展两项。

2 问题分析与总体设计

2.1 从数据依赖确定并行边界

设原图为 I、输出为 O。O(x,y)只读取 I 中以(x,y)为中心的九个像素。输入从读取完成后保持不变，不同任务负责互不重叠的输出行。因此，相邻任务即使读取同一输入行，也没有读写冲突；无需给像素读取或计算加锁。

必须区分“无冲突的输出计算”和“有冲突的任务分配”。静态方案没有共享领取状态；任务池方案需要多个线程共同更新下一个任务编号 next，这才是互斥或原子操作应该保护的对象。所有方案都在并行区域结束时汇合，避免主线程提前读取未完成结果。

2.2 用 PCAM 组织设计

课程中的 PCAM 将设计分为划分、通信、组合和映射[2]。本文据此作出以下选择。

划分：以输出像素为独立工作，沿行方向建立任务。通信：输入只读共享，任务读取真实邻行，不复制或交换整幅图像。组合：将连续像素合并成行块，避免逐像素领取任务。映射：比较预先均分任务与运行时按需领取。

选择连续行块是因为图像按行连续存储，便于顺序访问；每块最多 64 行，在 4094 个内部有效行上形成 64 个任务，4 线程时平均每线程 16 块，既保留重新分配的余地，又避免每个像素都访问同步变量。64 行是预先固定的实验参数，不声称它是最优粒度。

2.3 程序模块与数据流

I/O 模块读取 8 位 P5 PGM；串行与并行模块执行共同的单行 Sobel 公式；验证模块使用独立参考实现；基准模块负责计时与结果校验；Python 脚本汇总 CSV 和绘图。图像分配、文件读写、结果比较和绘图均与核心计时分离。

3 任务划分与并行实现

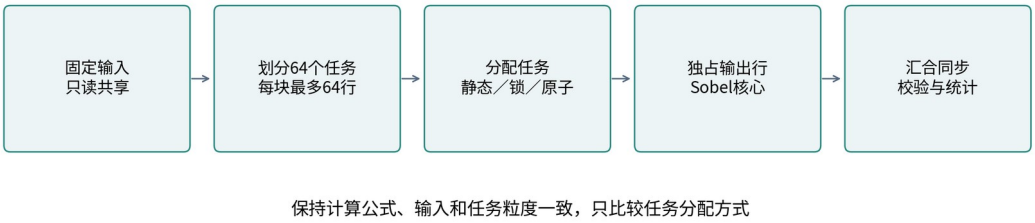


图 1 程序流程与控制变量

Fig. 1 Program workflow and controlled variables

3.1 一致的 Sobel 计算规则

水平与垂直卷积核分别为 Kx=[−1,0,1;−2,0,2;−1,0,1]、Ky=[−1,−2,−1;0,0,0;1,2,1][5]。内部像素输出为 floor(min(255, $\sqrt{(Gx^2+Gy^2)}$))，不做额外归一化或二值化。梯度以 int 累加，平方和转 double 后开方；最外圈输出直接为 0。

每次调用覆盖全部输出，包括边界；输入输出使用不同缓冲区。四种实现复用相同的单行函数，串行版也顺序遍历同一组 64 行任务块。这样，方案间的工作量和输出定义保持一致，差异集中于任务映射和同步。

3.2 行块定义与接缝处理

令 H 为图像高度，B=64。内部行数 N=max(0,H−2)，任务数 K=ceil(N/B)。任务 k 的起始行为 1+kB，结束行为 min(H−1,1+(k+1)B)，采用左闭右开区间。4096 行输入对应 N=4094、K=64：前 63 块各 64 行，最后一块 62 行。

任务 k 读取自身输出区域上下各一行真实输入。相邻块的输入读取可以重叠，输出写入不能重叠。任务接缝不是图像边界，不能在接缝处置零或截断邻域，否则即使各线程互不冲突，也会生成错误的边缘。

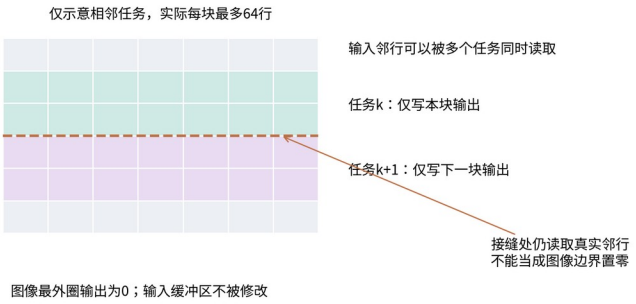


图 2 相邻任务的输出所有权与输入邻行

Fig. 2 Output ownership and neighboring input rows

3.3 四个版本的设计职责

版本	任务映射	共享状态与同步	设计目的
S 串行	按编号依次处理全部块	无工作线程	提供真正的串行基准
P1 静态	启动时计算连续任务区间	结束时隐式屏障	低分配成本，显式任务所有权
P2 锁任务池	完成一块后继续领取	锁保护 next；结束时屏障	应对实际执行速率差异
P3 原子任务池	与 P2 相同的按需领取	atomic capture 更新 next	比较单整数共享状态的实现选择

3.4 静态映射的设计与正确性

在 OpenMP parallel 区域中，线程 t 依据实际团队大小 p 取得任务区间[floor(Kt/p),floor(K(t+1)/p))。相邻区间首尾相接且互不重叠，所有区间的并集恰好是[0,K)，所以不存在遗漏或重复。这也能处理 K 不能被 p 整除、甚至 K 小于 p 的情况。

静态方案只在启动时计算范围，不必每块访问共享调度变量。对于计算量均匀、线程执行速率近似一致的环境，这是合理的低开销设计。但均分任务数并不保证同时完成：相同工作量的线程仍可能受到不同的实际调度、频率或资源竞争影响。

3.5 锁任务池：只保护共享领取状态

任务池不存放大量对象，只用共享整数 next 表示尚未领取的首个任务。每次核心调用将 next 置 0 并初始化一把 OpenMP 简单锁。线程循环执行以下过程：

加锁 → task=next 并递增 next → 解锁
若 task≥K 则退出
否则在锁外计算该任务，再次领取

如果不保护“读取并递增”，多个线程可能同时读取同一编号，造成重复任务和遗漏任务。互斥将领取操作串行化，保证成功领取的编号唯一。线程拿到编号后立即释放锁，整个行块的 Sobel 计算不会占有该锁。

线程仅在持锁时更新 next，不在锁内等待其他线程、不做 I/O，也不嵌套申请其他锁。并行区域汇合后销毁锁，确保没有线程仍使用它。这种结构避免了循环持锁等待；它不意味着任意修改后的程序都不可能死锁。

动态方案的机会在于较早完成的线程可以继续领取任务，减少固定分配造成的尾部等待。代价是每块领取与锁竞争。全图有 K 次成功领取，每个线程另有一次越界领取用于退出，因此总领取次数为 K+p；4 线程下为 68 次，而不是每像素一次。

3.6 原子任务池：适用条件与限制

P3 保留 P2 的任务池逻辑，只把“加锁—取值递增—解锁”替换成 OpenMP atomic capture，即以一个原子操作取得旧编号并递增共享整数[3]。像素计算仍在操作之外，所有任务完成后仍需并行区域末尾的同步。

这种替换成立的前提是共享领取状态仅为一个整数。若真实队列还需同时维护链表、队列头尾或任务状态，单个原子递增不能自动保证整个复合操作正确。原子操作也会竞争同一个共享位置，不能先验断言其整体程序必然比锁版本更快。

3.7 共享变量、局部变量与屏障

输入和输出对象共享，next 及锁在任务池中共享；线程编号、块编号、循环索引和梯度变量均为线程局部。使用 default(none)显式约束共享关系。各任务无需等待其他任务的中间结果，不设置块间屏障；并行区域末尾的隐式屏障则不可省略。

这体现了互斥与汇合的不同职责：锁或原子操作解决“谁领取哪个任务”，屏障解决“所有结果何时可被统一使用”。不在像素计算中加锁，是依据数据依赖作出的设计决定，而不是遗漏同步。

3.8 分析模型与适用范围

各版本计算复杂度均为 O(WH)，主要图像缓冲区空间为 O(WH)。动态分配额外执行 O(K+p)次领取。可用 Tp≈T 边界+T 最慢线程工作+T 分配与汇合描述实际时间，并结合含并行开销的 Amdahl 模型理解加速上限[4]。

增大并行度可能缩短像素计算，却不会消除分配、同步与资源争用。本实验不拟合串行比例，也不把所有非理想加速都归因于锁；这些因素需要依靠实测现象和适用范围分别讨论。

4 实验设计：测什么，为什么这样测

4.1 实验环境与固定输入

表 1 实验环境与不变条件

Table 1 Environment and fixed conditions

项目	配置
处理器与运行环境	Intel Xeon Platinum 8573C；KVM 云端虚拟环境
可见 CPU 与配额	5 个逻辑 CPU；cpu.max=400000 100000，等效 4 核
系统与内存限制	Debian GNU/Linux 13，x86_64；32 GiB
编译器与运行时	G++ 14.2.0；libgomp；_OPENMP=201511
统一编译选项	-O3 -std=c++17 -fopenmp -Wall -Wextra -Wpedantic
输入与任务粒度	4096×4096，8 位灰度；64 行/块，64 个任务
OpenMP 配置	DYNAMIC=FALSE；PROC_BIND=close；PLACES=cores；WAIT_POLICY=PASSIVE
测量重复与顺序	每配置预热 3 次；正式 30 轮；每轮打乱 10 种配置
线程数与实验范围	1、2、4 线程；只比较方案和线程数，不改变图像尺寸

固定输入由脚本生成，包含线性灰度背景、矩形、圆、三角形和条纹，不是自然图像数据集。它提供可视轮廓与可复现输入；Sobel 对每个内部像素执行相同公式，不依据边缘内容跳过计算。所有组共用一份 PGM，文件随提交保存。

输入 PGM 的 SHA-256：7630efcd74453761e2c346189ee5ca6c2e348c7d4854f0c89ab10aabee4e0ced

4.2 两项实验的自变量与控制变量

表 2 实验矩阵与设计理由

Table 2 Experimental matrix and rationale

实验	变化因素	比较对象	需要回答的问题
一：设计方案	分配与同步方案	S，以及 P1/P2/P3 各 4 线程	哪种设计在本环境下更有效?
二：线程扩展	线程数；分别观察各方案	P1/P2/P3 各 1、2、4 线程	扩展趋势是否取决于分配方式?

固定图像尺寸可把性能变化与工作规模变化分开，属于固定规模的强扩展考察。固定 64 行任务块可避免把任务粒度与同步方式同时改变；不比较动态调度的不同 chunk 参数，不增加其他图像算法。4 线程由有效 CPU 配额预先确定，不把超出配额的 8 线程作为主要测试点。

实验一比较三种并行设计，而不只比较“有没有线程”；实验二在每个设计内部观察线程增长。两项实验共享同一次测量中的 S、P1/P2/P3 数据，没有为了某个方案更换输入、编译参数或基准。

预期并不作为结论：均匀工作可能使静态分配占优，运行速率差异也可能使任务池占优。原子领取的临界操作更专门化，但整体耗时是否更短仍需测量。允许结果否定先验判断，才有比较设计方案的意义。

4.3 正式计时与统计设计

共 10 种配置：1 组串行与 3 方案×3 线程数。程序先验证实际线程团队可达到 1、2、4，再分别预热 3 次。正式阶段进行 30 轮，每轮用固定种子 20261025 打乱 10 种配置的顺序，共得到 300 条样本。打乱顺序用于减轻时间漂移对固定先后顺序的偏置，不保证彻底消除云端干扰。

所有配置使用同一个主线程上的 omp_get_wtime 计时[6]，计时包围完整核心调用及共同的配置分派。包括边界写入、实际像素运算、线程进入与退出、任务领取、锁初始化与销毁及汇合同步；排除输入输出、缓冲区分配、结果比较、统计和绘图。

预热可以减轻首次页访问和首次线程启动的影响，因此本报告描述预热后的重复运行表现，不把结果解释为首次运行或端到端图像处理性能。输出缓冲区每次均被完整覆盖，不能用上次结果代替本次计算。

每组取 30 次算术均值，标准差使用 n−1 分母；保留全部样本并附中位数与极值。图中误差线是标准差，不是置信区间，不以误差线重叠与否代替显著性检验。

加速比 Sp=Tserial/Tparallel(p)，效率 Ep=Sp/p。统一使用独立串行实现的均值作分母，而非将 OpenMP 的 1 线程结果当作串行基准，符合课程的性能评价定义[4]。

4.4 诊断运行为什么独立进行

完成正式计时后，另对 9 个并行配置各诊断 3 次，共 27 次。记录每线程完成的块数、内部行数、领取次数以及进入工作阶段到完成领取循环的耗时。后者包含线程被调度出去的时间，不包含末尾屏障等待，也不是纯 CPU 计算时间。

任务访问计数使用原子更新，每个任务应恰好访问一次。诊断逻辑以编译期模板参数区分，正式版本不执行计数器更新和每线程时钟调用。这样避免诊断额外引入的同步成本污染主结果。诊断只用于解释现象，不能替代正式耗时。

对 4 线程，静态版本预期每线程 16 块，而任务池允许分配不均。这里的“不均”可能正是对线程实际执行速率差异的适应，不能仅凭任务数不相等就判定调度差。线程耗时差也不直接证明某个具体硬件瓶颈。

4.5 正确性验收与边界用例

验证在性能结论之前进行。独立参考实现显式遍历两个 3×3 系数矩阵，不调用待测单行函数。覆盖 11 种尺寸、5 类输入：全零、常数、水平斜坡、阶跃和固定种子随机图。高度 66、67、131、257 覆盖 64 行整块、尾块及任务分界。

每幅图先验证串行，再组合 3 方案×4 种线程数（1、2、3、4）×4 种块高（1、3、16、64），共 2695 次参考比较。这里改变块高只用于边界正确性验收，不加入性能实验。另执行 2640 次诊断所有权检查，验证任务唯一、行数正确及领取次数符合 K+p。

手算 5×5 斜坡图的中心梯度为 Gx=24、Gy=0，输出 24；另检查非法线程数、非法块高及输入输出别名被拒绝。普通构建和 ASan/UBSan 构建均通过。此类内存与未定义行为检查不能替代数据竞争检测；并发正确性仍依赖所有权分析、领取原子性及验收结果。

固定全图的 300 次正式输出与串行结果逐像素一致，27 次诊断输出也一致，且每个任务恰好访问一次。所有输出均为 4096×4096，最外圈为 0。图像展示只是直观补充，不用肉眼相似代替数值一致。

5 实验结果与机制分析

5.1 实验一：固定 4 线程的设计方案比较

表 3 串行与三种 4 线程实现的正式测量

Table 3 Formal measurements of serial and four-thread implementations

配置	均值/ms	标准差/ms	加速比	效率/%
串行	57.914	4.135	1.000	—
静态分块	28.866	2.170	2.006	50.16
锁任务池	19.610	1.786	2.953	73.83
原子任务池	21.021	2.571	2.755	68.88

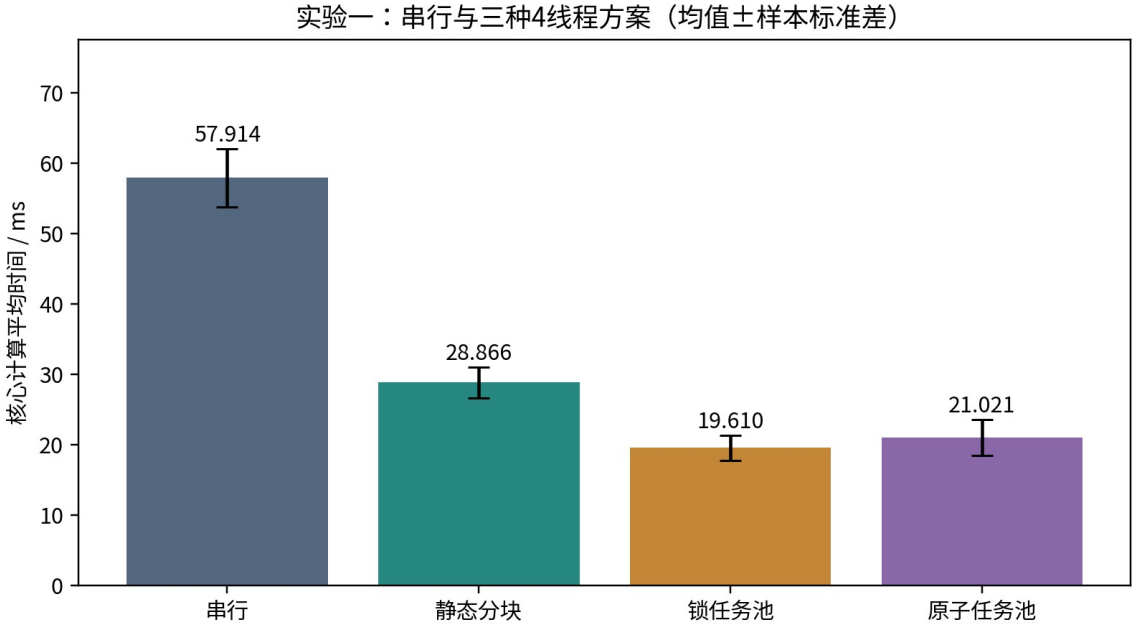


图 3 固定 4 线程时的方案比较（均值±样本标准差）

Fig. 3 Design comparison at four threads (mean ± sample SD)

三种方案相对串行均有平均加速。静态、锁和原子任务池的加速比分别为 2.006、2.953 和 2.755。锁任务池在本次测量中均值最小，相对静态分块缩短 32.06%，原子任务池相对静态缩短 27.18%。

这一结果说明，在当前环境和固定 64 行粒度下，按需领取任务所获得的收益超过了额外领取成本。但不能据此得出“锁比无锁计算快”：静态方案和任务池还改变了任务映射方式。更不能由两个程序的总时间差直接计算单次加锁或单次原子操作成本。

锁任务池比原子任务池的平均时间低 1.410 ms，而两组标准差分别为 1.786 和 2.571 ms。这里只报告本次整体实现的测量排序，不将其提升为锁原语性能的普遍结论。原子操作仍有共享位置争用，领取节奏、任务落点和云端波动都可能影响整体结果。

5.2 实验二：线程扩展与效率

表 4 各设计在 1、2、4 线程下的性能

Table 4 Performance of each design at one, two and four threads

方案	线程	均值/ms	标准差/ms	加速比	效率/%
静态分块	1	59.042	4.166	0.981	98.09
静态分块	2	31.127	5.667	1.861	93.03
静态分块	4	28.866	2.170	2.006	50.16
锁任务池	1	57.610	4.929	1.005	100.53
锁任务池	2	28.885	2.342	2.005	100.25
锁任务池	4	19.610	1.786	2.953	73.83
原子任务池	1	58.263	4.120	0.994	99.40
原子任务池	2	28.629	2.035	2.023	101.15
原子任务池	4	21.021	2.571	2.755	68.88

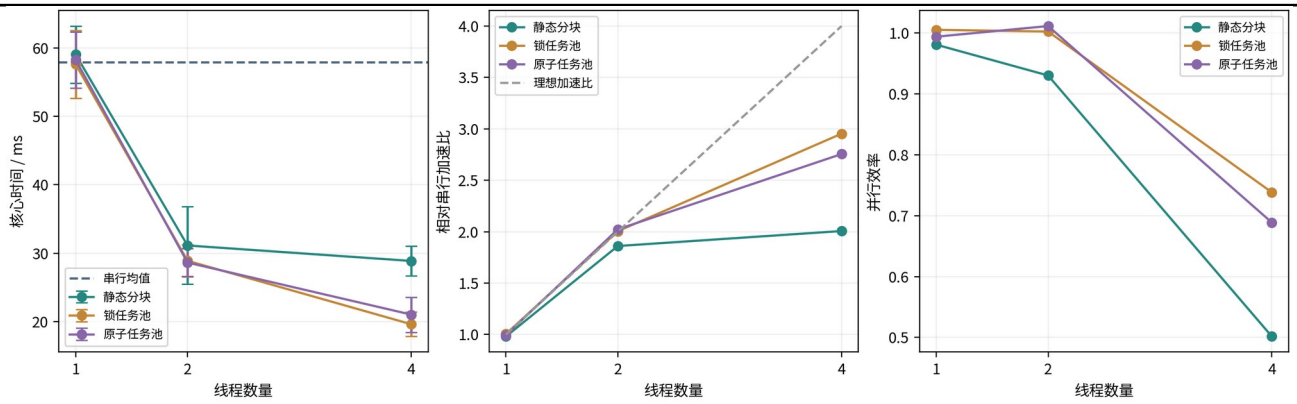


图 4 线程数量对核心时间、加速比与效率的影响

Fig. 4 Thread scaling of runtime, speedup and efficiency

三种设计从 1 线程增加到 4 线程，平均时间均下降，但扩展趋势不同。静态方案从 2 线程到 4 线程只进一步减少 7.27%，锁和原子任务池分别减少 32.11%和 26.58%。因此，仅给出一条“线程越多越快”的曲线会掩盖任务分配方式对扩展性的影响。

4 线程静态效率为 50.16%，锁任务池为 73.83%，原子任务池为 68.88%，均未达到理想线性加速。线程管理、尾部等待、领取成本和共享资源会消耗收益；当前实验不具备把这些因素逐项分离的证据。

部分 2 线程任务池配置的效率略高于 100%，1 线程锁版本也略快于串行。它们来自分别测得的均值，小幅超线性既可能涉及缓存状态，也可能受运行波动影响；不能将其解释为突破计算工作量约束。没有测量更多线程，不能声称 4 线程以上必然变慢。

5.3 诊断证据：均分工作为何不等于同时完成

表 5 第 1 次 4 线程诊断（不是正式计时均值）

Table 5 First four-thread diagnostic run, separate from formal timing

方案	线程	任务块	内部行	领取次数	线程工作/ms
静态分块	0	16	1024	0	18.630
静态分块	1	16	1024	0	29.902
静态分块	2	16	1024	0	30.015
静态分块	3	16	1022	0	13.791
锁任务池	0	21	1344	22	19.656
锁任务池	1	11	704	12	19.902
锁任务池	2	11	704	12	19.779
锁任务池	3	21	1342	22	19.654
原子任务池	0	21	1344	22	17.691
原子任务池	1	11	702	12	18.380
原子任务池	2	11	704	12	18.308
原子任务池	3	21	1344	22	17.578

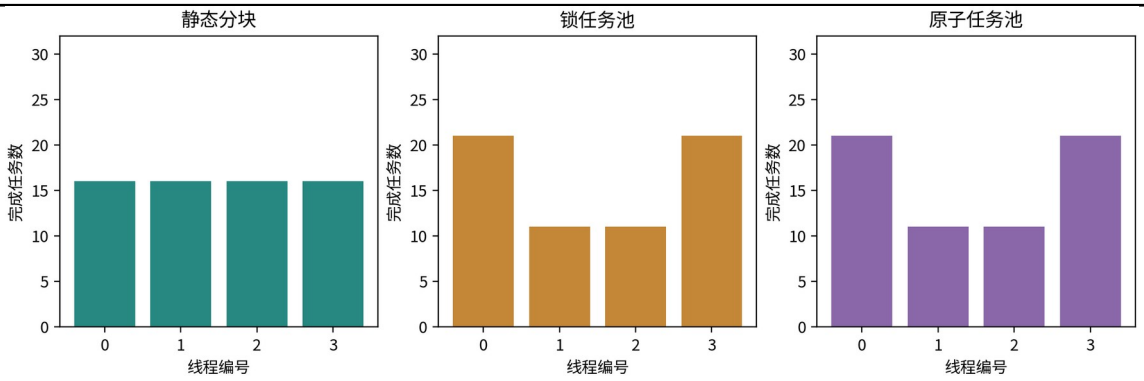


图 5 第 1 次 4 线程诊断的任务分配

Fig. 5 Task assignment in the first four-thread diagnostic run

静态版本中，4 个线程都完成 16 块，内部行数为 1024、1024、1024、1022，但工作阶段耗时约为 18.630、29.902、30.015、13.791 ms。行数仅差 2 行，无法从任务数量上解释如此大的时间差。并行区域仍需等待较慢线程结束，因此均分工作量没有消除尾部等待。

锁任务池在同一次诊断中的任务数为 21、11、11、21，线程工作时间约为 19.654～19.902 ms；原子任务池也出现按需调整的分配。较早完成的线程领取更多块，使完成时刻更接近。这与正式测量中任务池优于静态的现象相符，但不是对所有硬件环境的因果证明。

三次诊断的全部记录均已保留。第 2 次锁任务池诊断中仍存在明显时间差，说明动态分配不会消除所有干扰，也说明不能只选一幅诊断图概括稳定性能。诊断包含额外计数操作，其时间不可与表 3、表 4 的正式均值直接拼接。

5.4 环境约束与不能下的结论

正式计时窗口前后，cgroup 的 nr_throttled 均为 63，throttled_usec 均为 5016134，没有新增配额节流。配额用于确定最多 4 线程的主实验配置，但本次静态方案的性能差异不能直接归因于触发了容器限流。

诊断能够观察到线程有效完成速率的差异，却未测量 CPU 频率、宿主机负载、硬件缓存事件或线程被暂停的具体原因。因此，不能确认“某个 CPU 较慢”或“内存带宽已饱和”。close 绑定与 cores 位置保持固定，减少了一个配置变量，但不保证各工作线程拥有相同的独占物理资源。

同样不能声称原子任务池在所有环境下逊于锁任务池。若要研究同步原语的纯成本，需要独立设计微基准；若要证明 64 行最优，需要改变任务粒度。两者均超出本次两项实验，不能用未经测量的推论补成实验结论。

5.5 图像结果与验收对应

图 6 展示固定输入与三种 4 线程结果。矩形、圆和三角形轮廓均被提取；背景缓慢变化产生较弱响应。串行及 9 种并行输出的像素完全一致，分块接缝未引入额外边缘。该结论来自数值比较，缩略图仅便于阅读。

6 总结

本文以数据依赖为起点，把 Sobel 内部行组织为固定任务块，并设计静态映射、锁任务池和原子任务池。锁保护真实共享的任务编号，计算过程依靠输出所有权避免互斥，并在阶段结束时统一同步。设计过程对应任务分解、组合、映射及共享内存同步，而不是单纯设置线程数量。

本环境下，4 线程锁任务池取得 2.953 倍平均加速，静态分块取得 2.006 倍，原子任务池取得 2.755 倍。任务分配诊断表明，工作量均分不必然带来同时完成，动态领取可适应实际执行速率差异。结果同时保留了波动与诊断反例，没有把本次排序当作通用最优选择。

局限包括单一合成图、单一虚拟环境、固定任务粒度，以及缺少硬件计数器归因。所有结论限定于预热后的核心计算，不扩展到自然图像数据集、端到端 I/O、更多线程或其他算子。

7 参考文献

[1] 袁平鹏. 并行程序设计：课程介绍[Z]. 华中科技大学，2026，课件页内编号 58（考核要求）.

[2] 袁平鹏. 并行算法概述[Z]. 华中科技大学，2026，课件页内编号 48—49、74—81（PCAM、粒度与映射）.

[3] 袁平鹏. 共享内存程序设计[Z]. 华中科技大学，2026，课件页内编号 142、154—159、163（任务划分、同步与开发步骤）.

[4] 袁平鹏. 并行计算系统体系结构概述[Z]. 华中科技大学，2026，课件页内编号 35—38（加速比、效率与并行开销）.

[5] Huamán A. Sobel derivatives[EB/OL]. OpenCV 4.10.0, 2024[2026-10-25]. https://github.com/opencv/opencv/blob/4.10.0/doc/tutorials/imgproc/imgtrans/sobel_derivatives/sobel_derivatives.markdown.

[6] Free Software Foundation. GNU Offloading and Multi Processing Runtime Library[EB/OL]. GCC 14.2.0, 2024[2026-10-25]. https://github.com/gcc-mirror/gcc/blob/releases/gcc-14.2.0/libgomp/libgomp.texi.

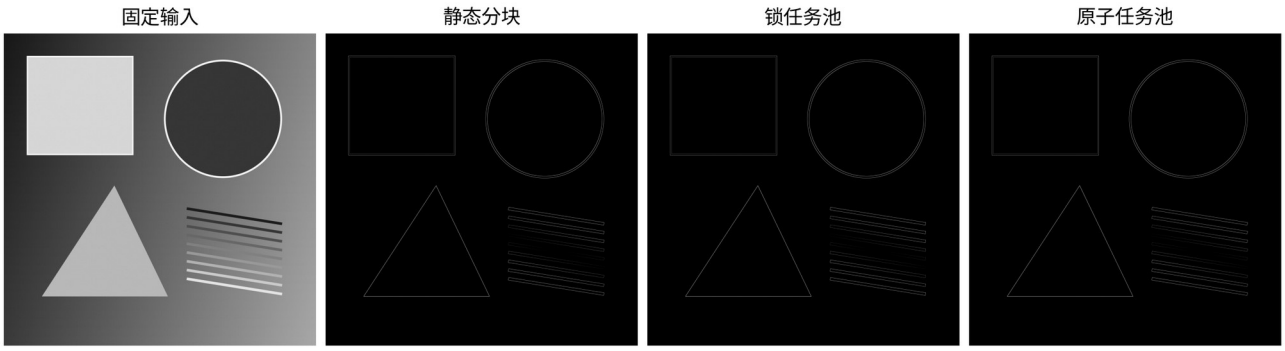


图 6 固定输入及三种 4 线程输出（原始全尺寸图随附）

Fig. 6 Fixed input and four-thread outputs; full-size images are included

附录：复现与提交文件

代码目录的 README 说明构建、单独运行、完整复现和报告生成方法。run_experiment.py 生成固定输入、执行测试、保存原始 300 条计时与 63 条线程诊断记录，再绘制两项性能图表。重新测量会覆盖结果，因此必须同步更新报告。build_report.py 直接读取本次汇总数据生成 DOCX，不手工录入性能数值。

实验结果目录保留输入 PGM/PNG、串行与全部 9 种并行配置的输出 PNG、原始及汇总 CSV、环境元数据、源文件校验值、配额计数、验证日志和图表。二进制、中间 PGM、旧版实验备份与原始模板放在仓库根目录 temp 中，不进入提交压缩包。

学号、姓名及学院班级留空，待提交者填写。代码与报告分目录保存，提交包按“学号-姓名”命名。